Lecture 05: Python Basics (Chapter 3 & 4)

교재 Ch.

유준수 (Joon Soo Yoo, Ph.D.)

2026년 3월 12일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

3.4 List Basics (데이터 보관함)

3.4 리스트(List)란 무엇인가?

여러 개의 데이터를 하나의 변수에 순서대로 묶어서 관리하는 도구입니다.

- **정의:** 대괄호 `[]` 안에 쉼표(`,`)로 구분하여 작성합니다.
- **다양성:** 숫자, 문자열, 실수 등 **서로 다른 타입**을 한 리스트에 담을 수 있습니다.
- **빈 리스트:** `empty = []` 또는 `empty = list()`로 생성합니다.

리스트 생성 예시

```
names = ["Yongjun", "Jinsung", "Minseok", "Moud"]   # DAS Lab 조교진
mixed = [2, "Hello", 3.14, True]   # 다양한 타입 혼합 가능
```

리스트 인덱싱과 길이 (Indexing & len)

문자열과 마찬가지로 각 요소는 고유의 번호(Index)를 가집니다.

데이터	"A"	"B"	"C"	"D"
양수 인덱스 (Forward)	0	1	2	3
음수 인덱스 (Backward)	-4	-3	-2	-1

- **접근 방법:** `list_name[index]`
- **길이 확인:** `len(list_name)` 함수를 사용하여 요소의 개수를 반환합니다.

리스트의 핵심: 수정 가능성 (Mutability)

리스트는 문자열과 달리 생성된 후에도 내부 요소를 마음대로 바꿀 수 있습니다.

요소 수정하기

```
scores = [80, 90, 70]
scores[2] = 100    # 3번째 요소(index 2)를 100으로 변경!
print(scores)    → [80, 90, 100]
```

3.5 Tuple Basics
(수정 불가능한 리스트)

3.5 튜플(Tuple)이란 무엇인가?

여러 데이터를 순서대로 묶는다는 점은 리스트와 같지만, **생성 후 내용을 바꿀 수 없다**는 점이 가장 큰 특징입니다.

- **정의:** 소괄호 `()` 안에 쉼표`(,)`로 구분하여 작성합니다. (괄호 생략 가능)
- **다양성:** 리스트와 마찬가지로 여러 타입을 섞어서 담을 수 있습니다.
- **접근:** 0부터 시작하는 인덱스를 통해 접근합니다.

튜플 생성 및 접근 예시

```
das_const = (3.14, "Security", 2026)   # 다양한 타입 묶기
print(das_const[1])   → "Security"
```

데이터 보관함의 차이: List vs Tuple

리스트와 튜플의 비교

특성	리스트 (List)	튜플 (Tuple)
변경 가능성	**Mutable** (수정 가능)	**Immutable** (수정 불가)
기능(Method)	`append()`, `pop()` 등	추가/삭제 함수 사용 불가
성능	메모리 사용량 많음	**메모리 효율적, 속도 빠름**

보안 관점에서

프로그램 실행 중 절대 변해서는 안 되는 상수(Constants)나 설정값은 **튜플**로 관리하는 것이 '데이터 무결성' 측면에서 훨씬 안전합니다.

Chapter 4. Decisions

4.1 Boolean Values

Branch: 프로그램의 선택 (Decisions)

- **Branch(분기):** 조건에 따라 실행할 코드 그룹을 선택하는 것입니다.
- 지금까지는 한 길로만 왔다면, 이제는 조건에 따라 다른 길을 갈 수 있습니다.

예시

"만약(If) 왼쪽 길이 더 넓으면 왼쪽으로 가고, 아니면(Else) 오른쪽으로 가라."

4.1 bool 데이터 타입

- **Boolean Value:** 단 2가지 상태인 `True`(참) 또는 `False`(거짓)만을 가집니다.
- 파이썬 키워드이며, **첫 글자는 반드시 대문자**여야 합니다.

변수 할당 예시

```
is_dessert = False    # Boolean 타입
is_fruit = "True"     # 따옴표가 있으면 문자열(String)입니다!
```

보안 관점에서

보안 시스템에서 `is_authenticated = True`는 접근을 허용하는 마법의 열쇠와 같습니다. 이 값이 원치 않게 바뀌지 않도록 관리하는 것이 중요합니다.

bool() 함수를 이용한 형변환

숫자나 문자열을 `True` 또는 `False`로 바꿀 수 있습니다. 규칙은 매우 간단합니다.

Boolean 변환 규칙

데이터 타입	False가 되는 경우	True가 되는 경우
숫자 (int, float)	0, 0.0	0이 아닌 모든 숫자
문자열 (str)	"" (빈 문자열)	최소 한 글자라도 있는 문자열

헷갈리기 쉬운 예시

`bool("0")` → `True` # "0"이라는 문자가 들어있기 때문!

`bool("")` → `False` # 아무것도 없기 때문!

비교 연산자 (Comparison Operators)

두 값을 비교하여 그 결과를 `bool` 값으로 반환합니다.

6가지 비교 연산자

연산자	의미	예시 (x=14)
`==`	같다 (Equal to)	`x == 14` → `True`
`!=`	다르다 (Not equal to)	`x != 14` → `False`
`>` , `<`	크다, 작다	`x < 13` → `False`
`>=` , `<=`	크거나 같다, 작거나 같다	`x >= 14` → `True`

주의 사항

`=` 는 값을 집어넣는 **할당(Assignment)**이고, `==` 는 같은지 물어보는 **비교(Comparison)**입니다.

4.2 If-else Statements

만약에... 라면? (조건부 실행)

if문: 조건이 참일 때만 실행

- **if statement:** 조건이 `True`이면 내부 코드를 실행하고, `False`이면 무시하고 넘어갑니다.

기본 구조

```
if condition:
# 조건이 True일 때 실행될 코드 (Body)
```

팁: Boolean 변수 그대로 쓰기

`if is_raining == True:` 라고 복잡하게 쓸 필요 없습니다.
`if is_raining:` 만으로도 충분합니다!

파이썬의 생명: 들여쓰기 (Indentation)

파이썬에서 들여쓰기는 단순히 보기 좋게 만드는 것이 아니라, **문법 그 자체**입니다.

- **권장 사항:** 한 단계 들여쓰기 시 **스페이스 4칸**을 사용합니다.
- 콜론(`:`) 다음 줄부터 시작되는 들여쓰기 된 모든 줄이 `if`문에 속하게 됩니다.

들여쓰기 오류 예시

올바른 코드	잘못된 코드 (Error)
`if age < 12:` `  print("Discount!")`	`if age < 12:` `print("Discount!")` *# 들여쓰기 누락*

if-else문: 두 갈래 길 (Either-Or)

조건이 `False`일 때 실행할 동작이 필요하다면 `else`를 사용합니다.

- **규칙:** 두 분기 중 **반드시 하나만** 실행됩니다. (둘 다 실행되거나 둘 다 안 되는 경우는 없습니다.)
- `else`는 `if`와 동일한 세로 라인에 위치해야 합니다.

Walking vs Driving

```
if distance < 1.0:
    print("We'll walk.")
else:
    print("We'll drive.")
```

Any Questions?